

ADA Lab Assignment 2

Algorithm Design and Analysis – Lab Experiment 2

Enrollment No: 2401420050 | Name: Dhruv Sharma

Experiment Overview

This lab experiment implements five core algorithmic paradigms:

- Task 1: Divide and Conquer – Merge Sort & Quick Sort
- Task 2: Sorting Performance Comparison
- Task 3: Greedy Algorithms – Activity Selection & Fractional Knapsack
- Task 4: Dynamic Programming – 0/1 Knapsack
- Task 5: Travelling Salesman Problem (TSP) – Brute Force

Task 1: Divide and Conquer

1.1 Algorithm Implementation

Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Quick Sort

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
```

```

left = [x for x in arr[1:] if x <= pivot]
right = [x for x in arr[1:] if x > pivot]
return quick_sort(left) + [pivot] + quick_sort(right)

```

Performance Timing

```

sizes = [100, 500, 1000, 5000]
merge_times = []
quick_times = []
for size in sizes:
    arr = [random.randint(1, 10000) for _ in range(size)]
    start = time.time()
    merge_sort(arr.copy())
    merge_times.append(time.time() - start)
    start = time.time()
    quick_sort(arr.copy())
    quick_times.append(time.time() - start)

```

1.2 Results

```

Input Sizes: [100, 500, 1000, 5000]
Merge Sort Times: [0.0, 0.00097, 0.00292, 0.02452]
Quick Sort Times: [0.0, 0.00099, 0.00195, 0.01464]

```

1.3 Observations

- Merge Sort has $O(n \log n)$ time complexity in all cases (best, worst, average).
- Quick Sort also achieves $O(n \log n)$ on average but can degrade to $O(n^2)$ in the worst case.
- In practice, Quick Sort is slightly faster due to lower constant factors.
- Both outperform $O(n^2)$ algorithms significantly for large inputs.

Task 2: Sorting Performance Comparison

2.1 Algorithms Implemented

- Bubble Sort – $O(n^2)$: Repeatedly swaps adjacent elements.
- Selection Sort – $O(n^2)$: Selects minimum and places it in order.
- Merge Sort – $O(n \log n)$: Divide and conquer merge approach.
- Quick Sort – $O(n \log n)$: Partition-based divide and conquer.

Bubble Sort Code

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

```

Selection Sort Code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        for j in range(i+1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
```

2.2 Timing Results

```
Bubble Sort: [0.00097, 0.01467, 0.02742, 0.11858]
Selection Sort: [0.0, 0.00681, 0.01171, 0.05889]
Merge Sort: [0.00096, 0.00193, 0.00291, 0.00488]
Quick Sort: [0.0, 0.00099, 0.00097, 0.00389]
```

2.3 Observations

- Bubble Sort and Selection Sort show dramatically increasing execution time as input grows – $O(n^2)$ complexity.
- Merge Sort and Quick Sort remain significantly faster due to $O(n \log n)$ complexity.
- For large datasets ($n=1000+$), Merge Sort and Quick Sort are the preferred choices.

Task 3: Greedy Algorithms

3.1 Activity Selection Problem

The greedy strategy selects the activity with the earliest finishing time at each step to maximize the number of non-overlapping activities.

Code

```
def activity_selection(start, finish):
    n = len(start)
    activities = list(zip(start, finish))
    activities.sort(key=lambda x: x[1])
    selected = [activities[0]]
    last_finish = activities[0][1]
    for i in range(1, n):
        if activities[i][0] >= last_finish:
            selected.append(activities[i])
            last_finish = activities[i][1]
    return selected

start = [1, 3, 0, 5, 8, 5]
finish = [2, 4, 6, 7, 9, 9]
```

```
result = activity_selection(start, finish)
```

Output

```
Selected Activities:  
(1, 2)  
(3, 4)  
(5, 7)  
(8, 9)
```

3.2 Fractional Knapsack

Items are sorted by value-to-weight ratio in descending order. Items are greedily picked; fractions are taken when capacity is insufficient.

Code

```
def fractional_knapsack(weights, values, capacity):  
    items = []  
    for i in range(len(weights)):  
        ratio = values[i] / weights[i]  
        items.append((ratio, weights[i], values[i]))  
    items.sort(reverse=True)  
    total_value = 0  
    selected_items = []  
    for ratio, weight, value in items:  
        if capacity >= weight:  
            capacity -= weight  
            total_value += value  
            selected_items.append((weight, value, 1))  
        else:  
            fraction = capacity / weight  
            total_value += value * fraction  
            selected_items.append((weight, value, fraction))  
            break  
    return total_value, selected_items  
  
weights = [10, 20, 30]  
values = [60, 100, 120]  
capacity = 50
```

Output

```
Selected Items (weight, value, fraction taken):  
(10, 60, 1)  
(20, 100, 1)  
(30, 120, 0.6666666666666666)  
Maximum Value: 240.0
```

3.3 Observations

- Activity Selection uses a greedy approach and produces an optimal solution.
- Fractional Knapsack achieves the maximum possible profit using ratio-based greedy selection.

- Both algorithms run in $O(n \log n)$ time (dominated by sorting).

Task 4: Dynamic Programming

4.1 0/1 Knapsack DP Table

The DP table is built bottom-up, storing optimal values for each combination of item count and capacity.

Code

```
def knapsack_01(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    return dp, dp[n][capacity]

weights = [10, 20, 30]
values = [60, 100, 120]
capacity = 50
```

Result:

Maximum Profit: 220

4.2 Recursive vs Dynamic Programming

Comparison of naive recursive approach vs DP for 0/1 Knapsack:

Code

```
# Recursive 0/1 Knapsack
def knapsack_recursive(weights, values, n, capacity):
    if n == 0 or capacity == 0:
        return 0
    if weights[n - 1] > capacity:
        return knapsack_recursive(weights, values, n - 1, capacity)
    return max(
        values[n - 1] + knapsack_recursive(weights, values, n - 1,
        capacity - weights[n - 1]),
        knapsack_recursive(weights, values, n - 1, capacity)
    )

weights = [1, 2, 3, 4, 5, 6, 7, 8]
values = [10, 20, 30, 40, 50, 60, 70, 80]
```

```
capacity = 15
```

Results:

```
Recursive Answer: 150
DP Answer: 150
Recursive Time: 0.08449 seconds
DP Time: 0.07476 seconds
```

4.3 Observations

- The recursive approach solves overlapping subproblems repeatedly, leading to exponential time complexity.
- Dynamic Programming stores intermediate results (memoization/tabulation), avoiding recomputation.
- Both approaches give the same correct answer (150), but DP is significantly faster for larger inputs.
- The DP approach has $O(n \times W)$ time complexity, where W is the knapsack capacity.

Task 5: Travelling Salesman Problem (TSP)

5.1 Brute Force Approach

The brute force TSP implementation checks all permutations of cities to find the minimum cost Hamiltonian cycle.

Code

```
from itertools import permutations

def tsp(graph, start=0):
    n = len(graph)
    cities = list(range(n))
    cities.remove(start)
    min_cost = float('inf')
    best_path = None
    for path in permutations(cities):
        current_cost = 0
        current_city = start
        for next_city in path:
            current_cost += graph[current_city][next_city]
            current_city = next_city
        current_cost += graph[current_city][start]
        if current_cost < min_cost:
            min_cost = current_cost
            best_path = (start,) + path + (start,)
    return min_cost, best_path

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
```

```
[15, 35, 0, 30],  
[20, 25, 30, 0]  
]
```

Results:

```
Minimum Cost: 80  
Best Path: (0, 1, 3, 2, 0)  
Execution Time: 0.00029 seconds
```

5.2 Exponential Growth Observation

The number of possible routes grows factorially with the number of cities:

Cities (n)	Possible Routes (n!)
2	2
3	6
4	24
5	120
6	720
7	5,040
8	40,320
10	3,628,800

5.3 Observations

- The brute force TSP has $O(n!)$ time complexity – it becomes infeasible for large n .
- For 4 cities, the algorithm checks 6 permutations to find the minimum cost tour.
- The optimal route found is: City 0 → City 1 → City 3 → City 2 → City 0 with cost 80.
- For $n \geq 15$, exact brute force becomes computationally impractical – heuristics or DP (Held-Karp) are needed.

Github: <https://github.com/Nighthawk-208/Analysis-And-Design-Of-Algorithms->